SNNLANG developer documentation

ar063 · 30 July 2026 · Draft

Contents

Developer guide

What snnlang does

snnlang is the network-authoring layer for *tools/snn*. It provides a small Python API for describing populations, projections, inputs, outputs, and recordings. Compilation checks the description and writes a portable, data-only bundle. *tools/snn* loads that bundle and performs the simulation or training.

The system separates four concerns:

1. The **graph** describes reusable computation.
2. The optional **training recipe** describes standard learning choices.
3. The **execution protocol** supplies data, duration, seeds, device, checkpoints, and recordings for one run.
4. The **experiment** owns the scientific question, conditions, analysis, figures, and conclusions.

A saved bundle contains scientific structure rather than live Python objects. It can be inspected and replayed without importing the authoring package.

Your first network

A network begins with an explicit timestep and a typed input. Components add reusable motifs. Outputs are values returned to a caller; observables are internal signals retained for inspection.

```
from tools import snnlang as snn

net = snn.Network("small_ping", dt=0.1 * snn.ms)
events = net.input(
    "events",
    shape=("time", "batch", 128),
    signal_type="spikes",
    unit="spike",
```

```
)

cell = snn.components.ping(
    net,
    name="cell",
    n_e=80,
    n_i=20,
    source=events,
)
net.expose(cell.E.spikes, cell.I.spikes, name="raster")

bundle = snn.compile(net, target="tools/snn")
bundle.write("small_ping.bundle", visualise=True)
```

Compilation produces `graph.json`, `manifest.json`, a readable report, and optional circuit diagrams.

Run the graph
Provide one tensor for every declared input:

```
import torch
from tools.snn.execution import ExecutionSpec, simulate

spikes = torch.zeros(2_000, 1, 128)
result = simulate(ExecutionSpec(
    kind="simulate",
    executor="graph",
    bundle="small_ping.bundle",
    inputs={"events": spikes},
    seed=42,
    device="cpu",
    recording="observables",
))

e_spikes = result.recordings["raster_0"]
i_spikes = result.recordings["raster_1"]
```

Graph-native forward simulation supports this example today. Input generation remains the caller's responsibility.

API reference

Package surface
Import the public authoring API with `from tools import snnlang as snn`. The top-level package exports `Network`, graph specification constructors, units, `compile`, `load_bundle`, `validate_graph`, and the `components`, `ops`, `readouts`, and `training` modules.

Minimal lifecycle

```
net = snn.Network(name, dt=0.1 * snn.ms)
bundle = snn.compile(net, training=None, target=None, assets=None)
root = bundle.write(path, visualise=False)
loaded = snn.load_bundle(root)
```

`Network` is mutable while authoring. `compile` returns a `Bundle` containing immutable graph data, optional training data, a manifest, diagnostics, and logical asset sources. `Bundle.write` returns the bundle directory. `load_bundle` verifies digests and validates the stored graph before returning a `Bundle`.

Execution boundary

Execution is deliberately separate from `tools.snnlang`. Import `ExecutionSpec`, `build`, `simulate`, `train`, or `infer` from `tools.snn.execution`. Graph execution is opt-in with `executor="graph"`; the legacy executor remains the default.

Collection reading path

Read the collection in this order:

1. **SNNLANG developer documentation.** This page establishes the architecture, first graph, execution boundary, and package lifecycle.
2. **Networks, signals, and parameters.** Core authoring objects, shapes, units, populations, and parameters.
3. **Components, projections, and delays.** Reusable motifs, connectivity, scheduling, and causality.
4. **Compiling and executing bundles.** Validation, bundle I/O, typed requests, and result objects.
5. **Inputs, outputs, and readouts.** Public outputs, recordings, standard readouts, and dense bindings.
6. **Training recipes and graph-native learning.** Declarative learning vocabulary and current execution limits.
7. **Runtime state, checkpoints, and provenance.** Continuation, compatibility, save/load, and replay identity.
8. **Compatibility, status, and extension.** Current support, remaining gates, diagnostics, and extension points.

Each page begins with its own linked contents, presents the high-level developer guide, and ends with the API reference.

Next: Networks, signals, and parameters